

Feature: Booking links, resources and availability rules

From build-plan: feature 2

Branch: `feature/booking-links-resources-and-availability-rules`

Status: verified

Approved step by step (decided 2026-09-25; see `AGENTS.md` , "A spec is approved one step at a time"): each of steps 2.1 to 2.6 got Frank's yes just before it was built. Verified 2026-09-28 by `/complete` 's final safety pass. Rewritten 2026-09-25 to match version 8 of the booking model, which Frank approved that day (the build log's "How it all fits together", and `project-plan.md` decisions 20 to 29). Where this spec and an older note disagree, version 8 wins.

Goal

Give every business the three scheduling primitives the rest of the product reads: its booking links (one per service), its resources (the people and places that do or host the work), and its bookable hours (when customers can book). Serve them on a public, read-only route a stranger's browser can call, resolve bookable hours through one function every later item reuses, and make the frontend call the API through `hc<AppType>` so a broken contract stops the build instead of failing at runtime.

In scope

- Three tables in `packages/shared` : `resource` , `booking_link` , `availability_rule` , with one migration.
- A resource is **one person or one place** (a room, a chair), never a group, and a column says which.
- **Every business has a first person, made automatically:** for every business that already exists, in the migration; for every new one, in a Better Auth hook. It is the business owner, named after the business.
- A booking link carries its length and **a buffer before and a buffer after**, any number including zero.
- `availability_rule` holds **bookable hours**: when customers can book online, not opening hours and not time at work. Unique per organization **and** resource. A null `resourceId` is the business's own row; a set one is that person's. The null-safe uniqueness is enforced by the database, not by code (see Data / contracts).
- A person's week is their own if they have one, otherwise the business's. A person can have **one-off dates** (hours on one date) without copying the week.
- **Set once per business, on the business's row only:** time zone, minimum notice, how far ahead customers can book, closed dates, the country and province, and the holidays the owner picked to close. The database refuses them on a person's row, so there is nothing to copy and nothing to drift.

- **Closures are for everyone, openings beat them:** a closed date or holiday closes online booking for everyone, whatever their own week says; a one-off date on it opens it again, for that person, or for everyone when it is the business's.
- A resource's rule can only point at a resource of the **same** organization, enforced by a composite foreign key.
- Zod validation schemas for a weekly-hours value, a one-off date and a whole rule, in `packages/shared`, used by every writer (the seeds now, settings in item 12).
- `resolveBookableHours`: the one function that works out, for a business and optionally one person, the week, the one-off dates, the closed dates within the horizon, and the business's settings.
- Two public routes, keyed by organization slug: `GET /public/:slug/booking-links` and `GET /public/:slug/booking-links/:bookingLinkId`.
- A public CORS rule for those routes, separate from the dashboard's, that never allows credentials, with origins from `WIDGET_ORIGINS`.
- The dev seed extended so both dev businesses have bookable data: the made-up painting company and clinic approved in 2.3. (A command that wrote links and hours into a real client's business was dropped on 2026-09-25: a client's hours are always set by the client in the app. See Out of scope.)
- The typed seam: the backend exports `AppType`, the frontend builds its client with `hc<AppType>` from `hono/client`, `fetchMe` moves onto it (its hand-written types go), and the dashboard home lists the business's active booking links through the public list route.
- Correcting the coding standard that says `AppType` lives in `packages/shared`. It cannot: see Notes for the AI.

Out of scope

- Computing bookable slots, and `GET /public/:slug/availability`. Items 5 and 9.
- Who does what (skills and rooms), the commitments table, standby and off, and the customer picking a person. Item 5; the owner ticks and sets them in items 12 and 12b.
- Linking a person to a login. Nothing in this item reads the link; the first item that does (item 3's per-person Google, or item 12b's calendars) adds it as one nullable column.
- The holiday picker (all the province's main holidays in one click, or one by one), and the one-click close and open screens. Item 12. Here nothing is closed by default; the seeds write the picks, closed dates and one-off dates.
- Any write route or settings screen for links, resources or hours. Item 12. Until then the dev seed is the only writer, and only on the local database.
- Any command that writes links or hours into a real client's business. Dropped by Frank, 2026-09-25: hours are always the client's own, set in the app; even when a new client asks for help with setup, it is done with them in Settings. The one gap this leaves, the agency's own business going live in item 10 before Settings exists in item 12, is noted on item 10.
- Calendar connection and free/busy. Item 3.

- Bookings, contacts, leads. Items 4 and 5.
- Per-resource screens. Nothing builds one until a tenant has two people.
- React Query. It arrives with the first screen that needs caching; this item's one list reads once on page load.
- Rate limiting the public routes. Recorded as a risk, not built.

Build loop

`workflow.stepReview` is `every` and `workflow.checkpointCommits` is `enabled`. Each step is committed and pushed without asking (Frank's standing yes for feature branches, 2026-09-26, in `AGENTS.md`), then gets `/audit` scoped to the step and the independent review, and P0/P1 findings are fixed or accepted before the next step starts. Each step's commit message starts with its number (`feat: 2.1 ...`). The build log is republished at every step. `/complete` makes the final commit and asks before merging.

Build steps

- ☒ **2.1 The three tables, the first person, and their validation.** Add `resource`, `booking_link` and `availability_rule` to `packages/shared/db/drizzle-schema.ts` exactly as in Data / contracts, with a comment beside the two uniqueness indexes saying why they are two (a future reader will otherwise "simplify" them into the broken single constraint), and one beside the check that keeps business settings off a person's row. Add `weeklyHoursValidationSchema`, `dateHoursValidationSchema` and `availabilityRuleValidationSchema` under `packages/shared/zod-validation/availability/`, exported through the existing `index.ts`. Generate the migration from `packages/shared`, add by hand the one statement that gives every existing organization its first person (kind `person`, named after the organization), and read the SQL before applying it. Then add `organizationHooks.afterCreateOrganization` to the organization plugin in `backend/lib/auth/auth-server.ts`, creating the same first person for every new business. It runs after the business is saved, not in the same transaction, which fails safe: a business without a person cannot take a booking. **Done when:** `db:generate` produces one migration whose SQL contains the partial unique index `WHERE "resourceId" IS NULL`, the composite unique index, the composite foreign key, the check on business-only columns, and the backfill; `db:migrate` applies it to `scheduleads_dev` and every existing organization then has exactly one resource; creating a business through the API as the platform admin gives it its person; and by hand in `psql`, four inserts are refused: a second business row for one organization, a second rule for one resource, a rule naming another organization's resource, and a person's rule carrying a time zone. The Vitest tests for the three validation schemas pass (added 2026-09-25, when the test runner went in). Both builds pass. **Approved by Frank, 2026-09-25**, point by point: the three tables and the four refusals; the first person starts with the business's name and the owner decides the final name (renamed in settings, feature 12); these checks as the proof.

- ☒ **2.2 The resolution function.** `backend/lib/bookable-hours/resolve-bookable-hours.ts` exports `resolveBookableHours( organizationId, resourceId | null, now )`. It reads the business's row and, when `resourceId` is set and that person has a row, the person's row. Both queries filter on `organizationId` first. It returns `ResolvedBookableHoursType` (Data / contracts), or `null` when the organization has no business row. The rules, in one place:
 - **Week:** the person's own, if their row has one; otherwise the business's.
 - **One-off dates:** the person's own. A person following the business's week also gets the business's one-off dates; on the same date, the person's own wins.
 - **Closed dates:** the business's closed dates (and holidays, from 2.6) inside the horizon, minus every date that has a one-off date for this person or for the business. A business opening a closed day opens it for everyone, each on their own hours.
 - **Time zone, notice, horizon:** always the business's. `now` is a parameter so the horizon is testable. The horizon runs from today in the business's time zone to today plus `horizonDays` (60 days on Sep 25 ends Nov 24); dates before or after it are dropped. **Split in two (Frank, 2026-09-25):** the rules are a pure function, `applyBookableHoursRules` in `backend/lib/bookable-hours/apply-bookable-hours-rules.ts`, with no database; `resolveBookableHours` only reads the rows and hands them over. The first version of this Done when said "against seeded rows", but the seeds are 2.3, so it could not be met in order. **Done when:** Vitest, in the backend, proves the rules half: a person's own week; the business's week for a person with no row; the business's week plus their date for a person with only a one-off date; on the same date the person's one-off date beats the business's; a business closed date closed for a person with their own week; a person's one-off date opening a closed day for them only; a business one-off date opening it for everyone; and past and beyond-horizon dates dropped. By hand, with rows added in `psql` and removed after, the database half returns `null` for a business with no business row, `null` for another business's person, and a real answer for a person with a row and one without. All tests pass, shared's included, and the backend build passes. **Approved by Frank, 2026-09-25**, point by point: the rules; the split and Vitest in the backend; this Done when.
- ☒ **2.3 The seeds.** Extend `packages/shared/scripts/seed-dev.ts` so, idempotently, the two dev businesses are made-up versions of the real client shapes (**approved by Frank, 2026-09-25**; they replace `agency-dev` and `test-salon-dev`, the two logins stay):
 - **Summit Painting (dev),** `painting-dev`, owned by `admin@example.com`, shaped like Primo: a business row with several windows a day and Sunday open; the owner (first person, an estimator), a second estimator and six painters; three booking links with buffers: interior estimate, exterior estimate, colour consultation. Crews and jobs join in feature 19.
 - **Riverbend Clinic (dev),** `clinic-dev`, owned by `owner@example.com`, shaped like Face and Body and bigger: a business row; six practitioners with mixed hours (some their own week, some following the clinic, one weekends only, one with only a one-off date); five rooms (kind `place`): rooms 1 and 2 massage only, room 3 massage and facials, room 4 massage and body treatments, room 5 laser only; ten of her real treatments with her real lengths (from `face-and-body/data/servicesData.ts`: facials, a peel, massages, laser, a body wrap), massages with 15

minutes after for room turnover. The room layout above is the contract for feature 5: which treatment needs which room, and who does what (at most four practitioners do massage, so massage never runs out of rooms; facials compete for room 3; laser has one room), are wired from it then, not invented again. Each business's first person is used, never duplicated, and **created by the seed when missing**: on a fresh database the 2.1 backfill runs before the seed makes any business, and the seed inserts businesses directly, so the Better Auth hook never fires (review finding F-17, 2026-09-25). The local database is rebuilt from scratch to prove it. The planned `db:seed-booking-link` command for real clients was dropped on 2026-09-25 (see Out of scope). **Done when**: the local `scheduleads_dev` is dropped and rebuilt from scratch (`db:migrate`, then `db:seed`; it only ever held seed data); Summit Painting then has its owner, a second estimator, six painters, three booking links and its business row, and Riverbend Clinic its owner, six practitioners with mixed hours, five rooms and ten booking links, each business with exactly one first person; `db:seed` run a second time changes no row count; every hours row is parsed by the 2.1 validation schemas before it is written, and the seed stops on a bad one; by hand, `resolveBookableHours` gives the weekends-only practitioner her own week and the practitioner with only an extra date the clinic's week plus that date; all tests and both builds pass. **Approved by Frank, 2026-09-25**, point by point: the two made-up businesses and the clinic's five-room layout; the real-client command dropped; this Done when, including rebuilding the local database.

- ☒ **2.4 The public routes.** Split `backend/server.ts` into `app.ts` (routes, exports `app` and `AppType`) and `server.ts` (only `serve`), so importing the type never starts a server. Add `backend/routes/public-booking-links-routes.ts` with the two routes in Data / contracts, mounted under `/public` with `backend/middleware/public-middleware/public-cors-middleware.ts`: origins from `WIDGET_ORIGINS` plus the dashboard origin, `credentials: false`, `GET` only. `WIDGET_ORIGINS` is already in `.env.example`. `refuse` in `backend/lib/errors/refuse.ts` gains `not_found` and `bad_request`. The org is found by slug, and the link only by `(organizationId, bookingLinkId)`, never by id alone. **A business whose plan does not include booking gets the same 404** (agreed with Frank, 2026-09-26): one more condition on the organization lookup, through `subscriptionIncludes`. **Done when**: saved Vitest tests in the backend call the app against the local seeded database (refusing any database that is not local or not `*_dev`, like the seed) and prove: both routes return the documented shape with no `organizationId` anywhere; an unknown slug, an unknown link, an inactive link, another business's link under this slug, a business with no business row, and a business whose plan lacks booking all return the identical `404` body; a malformed slug or id returns `400` in the same error shape; and no response carries `Access-Control-Allow-Credentials`. By hand, in a real browser, a `fetch` from an allowed origin reads the answer and one from a disallowed origin is blocked. All tests and the backend build pass. The backend tests then need the local Postgres running; the 2.2 tests stay pure. **Approved by Frank, 2026-09-26**, in two parts: what it builds (pieces 1 to 5, the plan-without-booking blocker first, answered yes); this Done when, as saved tests plus the browser by hand. **Built 2026-09-26**, all five pieces as planned: `app.ts` / `server.ts`, `routes/public-booking-links-routes.ts`, `publicCorsMiddleware`, `refuse` with `not_found` and `bad_request`, and two address schemas in `packages/shared/zod-`

`validation` (`organizationSlugValidationSchema`, `bookingLinkIdValidationSchema`). 19 saved tests in `backend/routes/public-booking-links-routes.test.ts`, three planted faults caught; the browser half checked by hand. No plan without booking exists until feature 23, so the test uses an unrecognised plan in its place.

- ☒ **2.5 The typed seam, proved.** Plan written and agreed Sep 27, both parts (piece 1, the blocker, answered yes). **Built 2026-09-27**, all five pieces as planned, plus `AGENTS.md` 's "not wired yet" line updated. By hand: the three links, the empty state, can't reach and its recovery, `/me` as before. Broken on purpose: the renamed route failed in `api-client.ts`, the renamed field in `booking-links-list.tsx`; both passed restored. Not pressed by hand: the list's own Try again (its state was seen once). 1. `hono ^4.13.8` in `frontend` for `hono/client` (needs a yes, Open question 1), plus the workspace link `"backend": ""`. 2. `backend/tsconfig.types.json` writes only the routes' declarations into `backend/dist/types/`; `backend/package.json` gets `build:types` and a types-only export `backend/app-type`. The **frontend's** `predev` and `prebuild` run it after the shared build. *Corrected Sep 27: the spec said "emitted by the backend build", but Vercel builds only the frontend, and old declarations left in `dist/` would let a renamed route pass the build. Declaration emit checked Sep 27: clean, status codes included.* 3. `frontend/lib/api-client.ts` builds two clients from one `AppType`: `dashboardApiClient` (sends the cookie) and `publicApiClient` (never does). *Corrected Sep 27 from "one client": 2.4's public CORS rule has `credentials: false`, so a credentialed request to `/public/` is blocked by the browser. `fetchMe` moves onto the dashboard client; `MeType` becomes the inferred type of `/me` 's 200 answer.* 4. `frontend/components/booking-links/booking-links-list.tsx` (new) takes the slug from `/me` and reads `GET /public/:slug/booking-links` once on page load: loading, list (name and length, name order), empty, not open for online booking yet (the public 404: no bookable hours until feature 12, or a plan without booking; no Try again, added by F-29 on 2026-09-27), and unreachable with Try again. `page.tsx` renders it under the business card. 5. Correct both `AppType` lines in `coding-standards.md` (File Organization and Data Fetching), adding the two-clients rule. Not in this step, said out loud: React Query (Out of scope), and the dev server does not pick up a renamed route's types until restarted. **Done when:** by hand, signed in as `admin@example.com`, the home lists `painting-dev` 's three links in name order; switched off for a moment in the local database, the empty state shows; with the API stopped, the unreachable state shows and Try again recovers; `/me` shows the business card as before and signed out still redirects to sign in. Then, broken on purpose: the list route renamed makes `npm run build --workspace=frontend` **fail with a type error** in `api-client.ts`, and `durationMinutes` renamed makes it fail where the list reads it; each passes once restored. All four outputs go on the step's page. Saved tests, both builds and lint pass. No new saved tests: the frontend has no test runner, and adding one needs a yes.
- ☒ **2.6 Holidays the owner picks.** The business's row says its country and province (`holidayCountry`, `holidayRegion`) and, new, which holidays the owner has picked to close (`closedHolidays`). The picked holidays inside the horizon, in the business's time zone, join the closed dates in `resolveBookableHours`, where a one-off date opens them like any closed date. **Nothing is closed by default: we build the functionality, the client decides their schedule** (Frank, 2026-09-28). The picker screen is feature 12. **Plan, rewritten 2026-09-28, five pieces.** The first

version (2026-09-27) closed Alberta's nine statutory holidays for every business automatically, following version 8's "the whole year on by default". Frank reversed that: none closes until the owner picks it. 1. Blocker: the holiday source (Open question 2). Recommended `date-holidays` in `backend` only: every province, Easter days, kept current by its maintainers; about 11 MB and four helper packages. **Answered 2026-09-28: yes, the whole package, no `--pick`**. Added as `date-holidays ^3.37.0`. The plan's size was wrong: 11 MB was the package alone; with its helpers it is 43 MB on disk and twelve packages (moon tables, moment, lodash), none with install scripts. Measured in memory: about 9 MB alone, about 3 MB on top of the running API (~215 MB). `holidays2json --pick CA,US` was weighed and left out: it saves memory only, and needs a hook that fails quietly. 2. The owner picks; nothing is closed by default. A new column, `closedHolidays`, on the business's row: a list of holiday names (`"Family Day"`), empty = none closed, refused on a person's row like every business setting. Names, not dates, so a picked holiday lands on the right date every year. The names come from the province's list and the country's national one (an Alberta business can also pick National Day for Truth and Reconciliation, which is national only). "All the province's main holidays in one click" is the picker's, feature 12. One migration, generated from `packages/shared`. 3. `backend/lib/bookable-hours/closed-holidays.ts` (new, the only file that knows the source) gives the dates of the picked holidays between two dates; `applyBookableHoursRules` adds them to the closed dates before one-off dates open any; a horizon past New Year asks for both years. `resolveBookableHours` passes the three columns along. 4. A picked name or a province the source does not know throws, like a business row missing a setting: a business that silently lost its Christmas closure would take bookings on Christmas Day. No picks, or no country, means only its own closed dates. 5. The seed: both made-up businesses `CA` / `AB`. The painting company picks all nine statutory days (the one-click case); the clinic picks four by hand (New Year's Day, National Day for Truth and Reconciliation, Canada Day, Christmas Day: the pick-and-choose case). Set on existing rows with no picks too, so no machine needs a rebuild. The shared validation schema gains `closedHolidays`. **Done when:** saved tests on the rules with a fixed `now`: a business with no picks gets only its own closed dates; Family Day picked closes Feb 16 in 2026 and, in 2027 when February 1 is a Monday, Feb 15; all nine picked close exactly nine dates in a year; a national-only pick (Truth and Reconciliation, Sep 30) closes for an Alberta business; Heritage Day, not picked, stays open; a one-off date on a picked Canada Day opens it for that person only; a horizon past New Year gets both years; an unknown name or province throws; planted faults make the right tests fail. By hand, today: the migration applies to the local database, which refuses `closedHolidays` on a person's row; the painting company's public detail route lists Thanksgiving (Oct 12) and Remembrance Day (Nov 11); the clinic's lists Sep 30, Dec 25 and Jan 1 and not Thanksgiving; the seed run twice changes nothing the second time; the lockfile gains only the package and its helpers, and the frontend build does not contain it. All tests, both builds, lint and the format check pass. **Changed from the first Done when (2026-09-27):** it asked the public route to show Family Day and Canada Day, but no business books that far ahead (60 and 120 days end late November and late January), so those are proved by saved tests with a fixed date. **Approved by Frank, 2026-09-28**, in two parts: what it builds (pieces 1 to 5, the holiday-source blocker first, answered yes); this Done when. **Built 2026-09-28** (`5a30d47`), all five pieces as planned, plus one addition: picks need a country, refused by the database

(`availability_rule_holidays_need_country_check`) and the form check. Migration `0002_closed_holidays` . 8 saved tests on the rules and 4 on the form check, three planted faults caught. **Review fixes, 2026-09-28:** F-31, the horizon is at most 365 days (migration `0003`); F-32 carried to feature 12; F-33, this block and the Files list brought up to date.

Files / areas

- `packages/shared/db/booking-tables/` - three tables, one file each (split from `drizzle-schema.ts` on 2026-09-27)
- `packages/shared/migrations/` - `0001` (the three tables, plus the backfill), `0002_closed_holidays` , `0003_horizon_at_most_a_year`
- `packages/shared/zod-validation/availability-validation-schemas/` - three validation schemas, exported from `zod-validation/index.ts`
- `packages/shared/scripts/seed-dev.ts`
- `backend/lib/auth/auth-server.ts` (the first-person hook), `backend/app.ts` (new), `backend/server.ts` (reduced), `backend/routes/public-booking-links-routes.ts` and its test, `backend/middleware/public-middleware/public-cors-middleware.ts` , `backend/lib/bookable-hours/resolve-bookable-hours.ts` , `backend/lib/bookable-hours/apply-bookable-hours-rules.ts` and its test, `backend/lib/bookable-hours/closed-holidays.ts` (2.6), `backend/package.json` (`date-holidays`), `backend/tsconfig.json` , `backend/tsconfig.types.json`
- `frontend/lib/api-client.ts` , `frontend/app/page.tsx` , `frontend/components/booking-links/booking-links-list.tsx` , `frontend/package.json`
- `.env.example` , `blueprint/context/coding-standards.md`

Data / contracts

Ids are text from `randomUUID()` , as in `seed-dev.ts` . Timestamps are `timestamptz` . Every table cascades on organization delete.

resource - one person or one place, never a group.

Column	Type	Notes	---	---	---	<code>id</code>	text PK		<code>organizationId</code>	text, not null	FK	
<code>organization.id</code>		<code>name</code>	text, not null	the first person is named after the business		<code>kind</code>	text, not null, default <code>person</code>	check: <code>person</code> or <code>place</code>		<code>active</code>	boolean, not null, default true	
<code>createdAt</code> ,	<code>updatedAt</code>	<code>timestamptz</code> , not null, default now										

Unique (`organizationId`, `id`) , which exists only so `availability_rule` can reference both columns.

booking_link - a service; the handle a host site stores as `Service.bookingId` .

Column	Type	Notes	---	---	---	<code>id</code>	text PK		<code>organizationId</code>	text, not null	FK
<code>organization.id</code>		<code>name</code>	text, not null			<code>slug</code>	text, not null	unique per organization, same rule			

as the org slug | | `description` | text, nullable | | | `durationMinutes` | integer, not null | check `> 0` | | `bufferBeforeMinutes` | integer, not null, default 0 | check `>= 0` | | `bufferAfterMinutes` | integer, not null, default 0 | check `>= 0` | | `active` | boolean, not null, default true | inactive reads as absent publicly | | `createdAt` , `updatedAt` | timestampz, not null, default now | |

Nothing reads the buffers until item 5, which keeps them inside a booking's stored time.

`availability_rule` - bookable hours.

| Column | Type | Notes | | --- | --- | --- | | `id` | text PK | | | `organizationId` | text, not null | FK `organization.id` | | `resourceId` | text, nullable | null = the business's row | | `weeklyHours` | jsonb, nullable | required on the business's row; null on a person's row = follows the business's week | | `dateHours` | jsonb, not null, default `[]` | one-off dates, shape below | | `timezone` | text | business's row only; IANA name; every minute and date is local to it | | `minimumNoticeMinutes` | integer | business's row only; check `>= 0` | | `horizonDays` | integer | business's row only; check `between 1 and 365` (at most a year ahead, since 2.6's review, F-31); no default, a writer sets it | | `closedDates` | jsonb | business's row only; `YYYY-MM-DD` list | | `holidayCountry` | text, nullable | business's row only; ISO 3166-1 alpha-2; null = no holidays | | `holidayRegion` | text, nullable | business's row only; the province, e.g. `AB` ; needs a country | | `closedHolidays` | jsonb, not null, default `[]` | business setting (2.6): the holiday names the owner picked to close, `[]` = none, nothing closed by default; a person's row must keep it empty; picks need a country | | `createdAt` , `updatedAt` | timestampz, not null, default now | |

One check keeps the two kinds of row honest: when `resourceId` is null, `weeklyHours` , `timezone` , `minimumNoticeMinutes` , `horizonDays` and `closedDates` are all set; when it is set, `timezone` , `minimumNoticeMinutes` , `horizonDays` , `closedDates` , `holidayCountry` and `holidayRegion` are all null and `closedHolidays` is empty (2.6). A person's row therefore holds only their own week and their one-off dates, and changing a business setting changes it for everyone.

Uniqueness, both indexes required (`resource-model-proposal.md` section 4): a partial unique index on `(organizationId) WHERE "resourceId" IS NULL` for the business's row, and a unique index on `(organizationId, resourceId)` for per-person rows. A single `unique(organizationId, resourceId)` would allow two business rows, because Postgres treats NULLs as distinct. Foreign key `(organizationId, resourceId) -> resource(organizationId, id)` , cascading, so a rule can never name another business's resource.

`weeklyHours` - keys `mon` to `sun` , each an array of `{ startMinute, endMinute }` , integers `0` to `1440` , `endMinute > startMinute` , windows within a day not overlapping. A missing or empty day is closed.

```
{ "sun": [{ "startMinute": 570, "endMinute": 1020 }],  
  "mon": [{ "startMinute": 1020, "endMinute": 1170 }],  
  "wed": [{ "startMinute": 450, "endMinute": 510 }] }
```

`dateHours` - one-off dates. Each entry is a date and its windows, same window rules as a weekday, at least one window. It replaces that date's hours, and it opens that date if it is closed or a holiday. Dates

unique within a row.

```
[{ "date": "2026-10-13", "windows": [{ "startMinute": 540, "endMinute": 780 }] }]
```

Validation also requires `timezone` accepted by `Intl.DateTimeFormat`, `holidayCountry` matching `^[A-Z]{2}$`, and `holidayRegion` matching `^[A-Z0-9]{1,3}$` and only with a country.

ResolvedBookableHoursType (from `resolveBookableHours`)

```
{
  source: "resource" | "organization"; // whose week answered
  timezone: string;
  weeklyHours: WeeklyHoursType;
  dateHours: DateHoursType; // the one-off dates that apply, inside the horizon
  minimumNoticeMinutes: number;
  horizonDays: number;
  closedDates: string[]; // closed dates and holidays inside the horizon, minus opened
}
```

GET /public/:slug/booking-links - public, no cookie, read-only.

```
{ "bookingLinks": [{ "id": "...", "slug": "estimate", "name": "Painting estimate",
                     "description": "...", "durationMinutes": 30,
                     "bufferBeforeMinutes": 0, "bufferAfterMinutes": 15 }] }
```

Active links only, ordered by name. An existing business with no active links returns `{ "bookingLinks": [] }`.

GET /public/:slug/booking-links/:bookingLinkId

```
{ "bookingLink": { "id": "...", "slug": "...", "name": "...", "description": null,
                  "durationMinutes": 30, "bufferBeforeMinutes": 0, "bufferAfterMinute
                  "availability": { "timezone": "America/Edmonton", "weeklyHours": { }, "dateHours": [
                  "minimumNoticeMinutes": 240, "horizonDays": 60,
                  "closedDates": ["2026-12-25"] } } }
```

The availability is the business's own (`resourceId` null); choosing a person arrives with bookings in item 5.

Errors on both: `404 { "error": { "code": "not_found", "message": "..." } }`, identical for an unknown slug, unknown or inactive link, a link of another business, and a business with no business row,

so a stranger cannot tell them apart. `400` with the same shape for a malformed slug or id. This extends the existing `refuse` shape and `RefusalCodeType` rather than adding a second error shape. Neither route ever returns `organizationId`, `source`, or anything about members, users or people.

Testing

Vitest went in on 2026-09-25, before 2.1, as the Sep 19 decision said ("a test runner goes in before item 2"); Frank approved the install. The test gate applies. 2.1 adds tests for the three validation schemas, 2.2 for every resolution rule (it carries seven), and 2.6 for the holiday dates; each step's `Done when` includes its tests passing, and every step reruns all of them. 2.2 also gives the backend Vitest and its `test` scripts, with its first test. What a unit test cannot prove is still proved by hand: real SQL against `scheduleads_dev`, real HTTP against the running API, a real browser for CORS (curl does not enforce it), and the deliberate broken build for the typed seam. Final gate: the tests, `npm run build -- workspace=backend`, `npm run build --workspace=frontend`, `npm run lint -- workspace=frontend`.

Notes for the AI

- **Version 8 is the source.** The booking model was approved on 2026-09-25 after three audits; its decisions are `project-plan.md` 20 to 29 and the build log's "How it all fits together". Do not reopen them; if something here seems to contradict them, say so and stop.
- **Bookable hours are not opening hours.** They say when customers can book online. The owner can still place work outside them (item 12b), and a job never hides anyone from customers.
- **Plans over overview.** `project-overview.md` still says "the two tables", puts `resource` in item 4, and calls `availability_rule` one per organization. Build to the plans. `/overview` should be rerun.
- **The first person and the business owner.** The first person is the business owner. When the platform admin creates a client's business, item 3b creates it under the client's email so the client is its business owner; today `creatorRole: "owner"` would make the platform admin the owner instead. This item does not link the person to any login (Out of scope), so it cannot link the wrong one.
- **AppType cannot live in `packages/shared`.** It is derived from the Hono app instance, so shared would have to import backend and invert the dependency. The first repo learned this; the frontend takes a type-only dependency on the backend workspace, the standard Hono RPC arrangement. `coding-standards.md` still says shared; 2.5 corrects it.
- **The first repo declared `AppType` and never consumed it.** 2.5 does not close on "it typechecks". It closes on a build that fails when a route is renamed.
- **Port, do not copy, from `scheduleads` feature 2** (`../scheduleads`, branch `main`):
`backend/src/routes/booking-links.ts`, the contract in `packages/shared/src/api/booking-link-contract.ts` (its `backend/scripts/seed-booking-link.ts` is not ported: that command was dropped). What changed: slug-keyed routes, `startMinute` / `endMinute`, a resource dimension, one-

off dates, business settings only on the business's row, buffers on the service, horizon and province holidays, `timestampz`, this repo's `refuse` error shape and naming.

- **Tenant scope.** The public routes take the slug from the path because a stranger has no session; that is the one place an organization comes from the URL, and it is read-only. Look up the org by slug, then every other query filters on its id. Never look a link up by id alone.
- **CORS.** The dashboard rule sends credentials; the public rule never does. Keep them separate, as the comment in `server.ts` already warns.
- **No new packages without Frank's yes.** `hono` in `frontend` and any holiday source are both asks.

Open questions

1. **Answered 2026-09-27: yes.** `hono ^4.13.8` and `"backend": "*" added to frontend/package.json`; the lockfile gained only those two lines, one `hono` copy is shared, and `backend` links to the workspace folder. 2. **Answered 2026-09-28: `date-holidays`, the whole package.** Added to `backend` only as `^3.37.0`: 43 MB on disk with twelve helper packages, about 3 MB of memory on top of the running API. About a million downloads a week, since 2016; code ISC, data CC-BY-3.0 (credit only if the holiday list itself is republished). See step 2.6, piece 1. 3. **Answered 2026-09-25: yes.** `/tests` ran before 2.1 and installed Vitest in `packages/shared`, with a first test on the subscription limits that was shown to fail when the old `in` bug is put back. See Testing.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- The package lost its `src/` folder during this feature (`062b56b`): `db/`, `zod-validation/`, `subscriptions/`, `helpers/` and `migrations/` sit straight in `packages/shared`, and on Sep 27 the tables went one per file under `db/auth-tables/` and `db/booking-tables/`, re-exported from `db/index.ts`. `drizzle.config.ts` finds them with the glob `./db/-tables/-table.ts`; `db:generate` found no change after the split.
- `db/booking-tables/` holds the three new tables. `resource` is one person or one place (`kind`), with a unique (`organizationId`, `id`) that exists only so a rule can reference both columns. `booking_link` carries its length and a buffer before and after, each checked. `availability_rule` is bookable hours: a null `resourceId` is the business's row, a set one is a person's. Two uniqueness indexes, because one `unique(organizationId, resourceId)` would allow two business rows (Postgres treats NULLs as distinct), and a comment beside them says so. A row-kind check keeps

every business setting (time zone, notice, horizon, closed dates, country, province, picked holidays) off a person's row, and a composite foreign key keeps a rule from naming another business's resource.

- Migrations: `0001` creates the three tables and gives every existing business its first person (a hand-added backfill); `0002_closed_holidays` adds `closedHolidays` and the check that picks need a country; `0003_horizon_at_most_a_year` narrows the horizon check to 1 to 365 days.
- `zod-validation/availability-validation-schemas/` holds the weekly-hours, one-off-date and whole-rule schemas every writer uses (the seed now, Settings in feature 12). Windows are `startMinute` / `endMinute` in minutes from midnight and may not overlap within a day. Two address schemas (`organizationSlugValidationSchema` , `bookingLinkIdValidationSchema`) serve the public routes.
- `scripts/seed-dev.ts` builds Summit Painting (`painting-dev` , shaped like Primo: an owner-estimator, a second estimator, six painters, three links) and Riverbend Clinic (`clinic-dev` , shaped like Face and Body: six practitioners with mixed hours, five rooms, ten treatments). It is idempotent, parses every hours row through the shared schemas before writing, creates a business's first person when missing (the auth hook never fires for rows the seed inserts, F-17), and sets both businesses to `CA / AB` with their holiday picks, on existing rows too.
- Vitest arrived before 2.1 with a first test on the subscription limits; the availability schemas have their own tests.

backend

- `backend/src/` is gone (`51154ff`): `app.ts` , `server.ts` , `database.ts` , `lib/` , `middleware/` and `routes/` sit straight in `backend/` , grouped by area, one middleware per file, each named for what it does and ending in `Middleware` .
- `app.ts` holds every route in one chain and exports `app` and `AppType` ; `server.ts` only calls `serve` , so importing the type never starts a server. `tsconfig.types.json` writes only the routes' declarations into `dist/types/` , and `build:types` is run by the frontend's `predev` and `prebuild` , because Vercel builds only the frontend.
- `lib/auth/auth-server.ts` gained `afterCreateOrganization` , which gives a new business its first person, named after the business. It runs after the business is saved, not in the same transaction, which fails safe: a business with no person cannot take a booking.
- `lib/bookable-hours/` is split in two on purpose. `apply-bookable-hours-rules.ts` is pure and holds every rule: the person's week if they have one, otherwise the business's; the person's one-off dates beating the business's on the same date; closed dates and picked holidays closing for everyone; any one-off date reopening a closed day; the horizon counted from today in the business's time zone, never the server's. `resolve-bookable-hours.ts` only reads the rows, the business's first and every query filtered on the organization, and returns `null` for a business with no row or a person of another business.
- `lib/bookable-hours/closed-holidays.ts` is the only file that knows `date-holidays` . A pick is the package's English name, looked up in the province's list and the country's own, for every year the horizon touches. It throws on an unknown country, province or name, because the package silently

falls back to the national list for a bad province and a business that quietly lost its Christmas closure would take bookings on Christmas.

- `routes/public-booking-links-routes.ts` answers the two public routes. The business is found by slug first (it needs a plan that includes booking and a business row), and a link only by `(organizationId, bookingLinkId)`, never by id alone. Every miss gives the same `404` body, so a stranger cannot tell an unknown business from one without booking; a malformed slug or id gives `400` in the same shape. Neither answer carries `organizationId` or `source`. `lib/errors/refuse.ts` gained `not_found` and `bad_request` rather than a second error shape.
- `middleware/public-middleware/public-cors-middleware.ts` is `GET` only with `credentials: false`, origins from `WIDGET_ORIGINS` plus the dashboard's, kept apart from `dashboardCorsMiddleware`.
- `routes/public-booking-links-routes.test.ts` calls the real app against the seeded local `scheduleads_dev`, adds its own rows and removes them, and refuses any database that is not local and `*_dev`. Since 2.4 the backend tests need Postgres running; they fail rather than skip without it.

frontend

- `lib/api-client.ts` builds two clients from one `AppType`: `dashboardApiClient` sends the login cookie, `publicApiClient` never does, because the public CORS rule refuses credentials and the browser would discard the answer. `fetchMe` moved onto the dashboard client and `MeType` is now inferred from `/me`'s 200 answer; the hand-written copy is gone.
- `components/booking-links/booking-links-list.tsx` reads the business's active links once on page load through the public list route, with five states: loading, the list, empty, not open for online booking yet (the public 404, no Try again, F-29), and unreachable with Try again. `app/page.tsx` shows it under the business card.
- `hono` and the workspace link `"backend": "*"` were added to the frontend (a yes on Sep 27). Proved by breaking it on purpose: a renamed route and a renamed field each failed `npm run build -- workspace=frontend` with a type error.

Decisions worth knowing later

- Version 8 of the booking model (project-plan decisions 20 to 29) is the source; this feature built its first three primitives.
- Bookable hours are when customers can book online, not opening hours and not time at work.
- Nothing about a client's schedule is decided by the product (Frank, 2026-09-28, project-plan decision 30): no holiday is closed by default, and the planned command that wrote hours into a real client's business was dropped on Sep 25. The seeds are the only writer until feature 12's Settings.
- `AppType` lives in the backend, not `packages/shared`: it is derived from the app instance, so shared would have to import the backend.
- The one place an organization comes from the URL is the public routes, read-only, and every query after the slug lookup filters on its id.

- A business whose plan lacks booking gets the same 404 as an unknown one.

Carried forward, not done here

- F-12 (P2) and F-13, F-14, F-16 (P3) from feature 1 stay open in the live ledger. F-32 (P3, a pick stored as the package's display name) goes to feature 12, where the picker is built. F-34 (P3, three tsconfig comments carrying history) is new from the final review.
- The person branch of `resolveBookableHours` has no saved test and no caller yet; it was proved by hand in 2.2 and is first used in feature 5.
- The public routes have no rate limit (Out of scope, a known risk).
- The overview still describes the older two-table model; `/overview` should be rerun before feature 3 is spec'ed.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live.

2/F-15 [P3] closed - `coding-standards.md` still says migrations run through drizzle-kit from `backend`

File: blueprint/context/coding-standards.md:240 **Found:** 2026-09-23 by /audit (scope: current; lens: quality) **Why it matters:** The Backend section says Drizzle migrations "run through `drizzle-kit` from `backend`". This feature moved them: `drizzle.config.ts` and the `db:*` scripts live in `packages/shared`, and `AGENTS.md` (Commands) says "Never generate a migration from `backend` or `frontend`: two workspaces generating against one database is how a migration ledger forks." Same class as F-09, in the same file, missed by its repair. An agent obeying the standard would recreate the fork `AGENTS.md` warns about.

Suggested fix: Rewrite the sentence to say migrations are generated and applied from `packages/shared` with the commands in `AGENTS.md`. **Resolution:** 2026-09-23, during Frank's folder walkthrough. The Backend section now says migrations run through `drizzle-kit` from `packages/shared` only, never from `backend` or `frontend`, and points at Commands in `AGENTS.md`. Awaiting re-review. Closed 2026-09-27 by /audit independent (step 2.4, `e58e0a0`): `coding-standards.md:333-337` now says Drizzle schema and migrations run through `drizzle-kit` from `packages/shared` only, never from `backend` or `frontend`, matching `AGENTS.md` Commands and the `db:*` scripts in `packages/shared/package.json`. No other line in the file says otherwise.

2/F-17 [P2] closed - A database rebuilt from migrations and the seed has businesses with no first person

File: packages/shared/scripts/seed-dev.ts:87 **Found:** 2026-09-25 by /audit independent (scope: current; lens: quality) **Why it matters:** Step 2.1 makes "every business has a first person" an invariant and states it in `drizzle-schema.ts:138`: made by migration 0001 for businesses that existed then, and by the `afterCreateOrganization` hook for every new one. `seed-dev.ts` is a third way a business is created, and it inserts `organization` rows directly through Drizzle, so the Better Auth hook never runs. On a fresh database, the documented rebuild path (`AGENTS.md` : " `db:migrate` builds a fresh one and `db:seed` makes it usable"; the workspace rule that each machine rebuilds its local database from migrations and the seed) runs the migration's backfill against zero organizations, then the seed creates `agency-dev` and `test-salon-dev` with no resource. This machine's database hides it because both businesses existed before 0001 was applied (checked: one `person` each). Nothing reads `resource` yet, so nothing fails today, but step 2.3's plan says "The first person every business already has from 2.1 is used, never duplicated", which is false on any other machine. **Suggested fix:** In `seed-dev.ts`, after the organization is found or made, insert its first person (kind `person`, named after the business) when it has no resource, in the same transaction. This can land in step 2.3, which already edits this file, but correct the 2.3 plan text now so it says the seed creates the person when missing rather than assuming it exists. **Resolution:** Still open. Planned into step 2.3 with Frank on 2026-09-25: the seed creates each dev business's first person when missing; the spec's 2.3 text was corrected the same day. Closed 2026-09-26 by /audit independent (step 2.3, `7b04316`): `seed-dev.ts:281` now finds or makes the first person (kind `person`, named after the business) inside the seed's one transaction, before any hours or people. The builder left this entry `open` rather than `fixed`; this pass re-examined the repair directly. Evidence: the local database was rebuilt (both users, both businesses and all 20 resources carry one creation timestamp from the seed's transaction, and no old dev business remains), each business has exactly one resource named after it, and a second `db:seed` left every row of the six seeded tables byte-identical. No new defect in the repair; related gaps in the same file are recorded as F-22 to F-25.

2/F-18 [P3] closed - Several refinements in the business-row schema have no test

File: packages/shared/src/zod-validation/availability/availability-rule-validation-schema.ts:28 **Found:** 2026-09-25 by /audit independent (scope: current; lens: tests) **Why it matters:** The standard (`coding-standards.md`, Testing) asks for tests on validators where a wrong answer is possible. The 15 new tests cover the week, the one-off dates and four rule cases, but not the hand-written duplicate-closed-date refine (line 28), the `holidayCountry` and `holidayRegion` patterns (lines 31-40, e.g. a lowercase `ca` or a four character region), `horizonDays` at 0, or a negative `minimumNoticeMinutes`. Each mirrors a database check or a spec rule, so a regression there would only surface as a raw constraint error from a writer in 2.3 or item 12. **Suggested fix:** Add one refusal test per rule to `availability-validation-schemas.test.ts`, ideally asserting the issue path (`holidayRegion`, `closedDates`) rather than only `success === false`. **Resolution:** Fixed 2026-09-25 on Frank's yes: four tests added (a closed date twice, country and province written out instead of their codes, zero days ahead, negative notice). Removing the horizon and notice limits made exactly those two tests fail. Awaits the next review pass to close. Closed 2026-09-26 by /audit independent: the four tests are at `availability-validation-schemas.test.ts:104-124`. Each targets a row only the business branch could accept (the person

branch is `.strict()` and refuses a time zone), so removing the refine, either regex or either limit flips that test; 25 shared tests pass. Asserting the issue path was suggested, not required.

2/F-19 [P2] closed - No test catches a person with a row losing the business's one-off dates

File: backend/src/lib/availability-rules.test.ts:49 **Found:** 2026-09-25 by /audit independent (scope: current; lens: tests) **Why it matters:** Step 2.2's rule says "A person following the business's week also gets the business's one-off dates" (`current-feature.md`, 2.2), and `availability-rules.ts:67-69` does it. But no test gives a follower who has a row and the business one-off dates on different days. The "only a one-off date" test runs against a salon with no one-off dates, and the "same date" test only proves the override. Checked by mutation in a scratch copy: changing line 67 to `hasOwnWeek || person !== null` drops the business's one-off dates for every follower with a row, and all 10 tests still pass. That is the function items 5 and 9 reuse, and the wrong answer is silent: the person would be bookable on the week's hours on a day the business changed. **Suggested fix:** Add one test: the salon with a one-off date on one day, a follower with their own one-off date on another, and assert both dates come back, sorted. **Resolution:** 2026-09-25, fixed by /implement: added the test "a person with a row who follows the business's week keeps the business's one-off dates" (salon opens Nov 2, Ben has his own Oct 13, both come back sorted). Re-ran the reviewer's mutation (`hasOwnWeek || person !== null` on line 67): the new test fails by name, 1 failed and 10 passed; restored, 11 pass. Awaiting re-review. Closed 2026-09-26 by /audit independent: the test at `availability-rules.test.ts:57` gives a follower with a row (Oct 13) and a business one-off date (Nov 2) and asserts both, sorted. Under the mutation the follower gets only `personDateHours`, so the assertion fails; the mutation was traced, not re-run from a copy. 11 backend tests pass.

2/F-20 [P3] closed - `AvailabilityRuleRowType` is the one type in the project that is not exported

File: backend/src/lib/resolve-availability.ts:15 **Found:** 2026-09-25 by /audit independent (scope: current; lens: quality) **Why it matters:** `coding-standards.md` (Naming) says types end in `Type` "and always exported". Every other type under `backend/src` and `packages/shared/src` is exported (the only exceptions are the augmented `ContextVariableMap` interfaces the standard names). Small, but it is the pattern the next file copies. **Suggested fix:** `export type AvailabilityRuleRowType = ...`. **Resolution:** 2026-09-25, fixed by /implement: `export type AvailabilityRuleRowType`. Backend build and Prettier pass. Awaiting re-review. Closed 2026-09-26 by /audit independent: exported at `resolve-availability.ts:15`; backend build passes. The same class recurs in the 2.3 seed, recorded separately as F-24.

2/F-21 [P3] closed - The build log's "full diff" links move with the branch

File: blueprint/context/project-log.html:3715 **Found:** 2026-09-25 by /audit independent (scope: current; lens: quality) **Why it matters:** `AGENTS.md` says each code drawer "Ends with a link to the step's commit

on GitHub for the full diff". The 2.2 drawer links `compare/b4b39a5...feature/booking-links-resources-and-availability-rules`, and the 2.1 drawer (line 3328) `compare/cb8e09c...` the same branch. A compare against a branch name follows the branch head, so the 2.1 link already shows 2.1 and 2.2 together, and each new push widens both. After the merge they show the whole feature, not the step. **Suggested fix:** When the next step republishes the page, pin each link to the step's own commits (`compare/b4b39a5...2519644` for 2.2, `compare/cb8e09c...b4b39a5` or the step commit for 2.1). A step's own hash cannot be in its own commit, so pin it on the following republish. **Resolution:** 2026-09-25, fixed by /implement: the 2.1 link now compares `cb8e09c...ab16897` and the 2.2 link `b4b39a5...98e2493`, each a step plus its review fix. `AGENTS.md` now says every drawer link is pinned to commits, never the branch. Awaiting re-review. Closed 2026-09-26 by /audit independent: `project-log.html:3375` compares `cb8e09c...ab16897` and `:3817 b4b39a5...98e2493`, exactly 2.1 plus its fix and 2.2 plus its fix. No link in the page targets a branch name; the 2.3 drawer says its link comes once committed, as the F-21 note expects.

2/F-22 [P3] closed - The seed's one-off and closed dates are fixed dates, so the clinic loses its "only an extra date" practitioner after Oct 18

File: `packages/shared/scripts/seed-dev.ts:174` **Found:** 2026-09-26 by /audit independent (scope: current; lens: quality, tests) **Why it matters:** The seed promises "six practitioners with every kind of hours" (line 146), and the 2.3 contract and its spot check rely on Daniel being the practitioner with only a one-off date. His date is the literal `2026-10-18`. From Oct 19, 2026 `resolveAvailability` drops it as past, so Daniel resolves exactly like Ana, and a database rebuilt from migrations and the seed (the workspace rule for every machine) no longer shows that case at all. The closed dates (lines 97 and 144) run out on 2027-01-01 the same way, and Summit Painting's two closed dates already sit outside its 60-day horizon, so the painting business resolves with no closed dates until late October (checked with `resolveAvailability` on the seeded rows). Features 5 and 9 compute slots against this data.

Suggested fix: Compute the seed's dates relative to the day it runs (for example Daniel on the Sunday about three weeks ahead, closed dates inside each horizon), or say beside line 174 that the case expires and when to move it. **Resolution:** 2026-09-26, fixed by /implement on Frank's yes: the seed's dates are relative to the day it runs. Each business gets one closed day 21 days out (inside both booking windows) and Daniel's extra day is the Sunday at least 14 days out. Proved by rebuilding the practice database: closed day 2026-10-17 for both, Daniel 2026-10-11, a Sunday. Awaiting re-review. Closed 2026-09-27 by /audit independent (step 2.4, `e58e0a0`): `seed-dev.ts:57-66` computes `daysFromToday` and `sundayAfterDays` from the run date; both closed-date lists (`:112`, `:159`) are one day 21 days out, inside both horizons (60 and 120), and Daniel's date (`:189`) falls 14 to 20 days out, so it can never land on the closed day. A rebuilt database therefore always shows every case. The dates still freeze once written (the hours rows are find-or-make), which the comment at `:57-58` states: rebuilding refreshes them, as the original suggested fix allowed.

2/F-23 [P3] closed - A seeded person given one-off dates but no `weeklyHours` key is silently skipped

File: packages/shared/scripts/seed-dev.ts:313 **Found:** 2026-09-26 by /audit independent (scope: current; lens: quality) **Why it matters:** `PersonSeedType` makes both `weeklyHours` and `dateHours` optional (lines 60-61), but line 313 writes a row only when `weeklyHours !== undefined`. An entry like `{ name: "Ana", kind: "person", dateHours: [...] }` type-checks and its dates are dropped without a word. Daniel works only because he spells out `weeklyHours: null`. This file is the only writer of hours until item 12, and the next person to give a follower a one-off date will write it the natural way. **Suggested fix:** Decide the row from both fields (`weeklyHours !== undefined || dateHours !== undefined`), or make the type say it, for example one optional `hours: { weeklyHours, dateHours }` whose presence means a row. **Resolution:** 2026-09-26, fixed by /implement on Frank's yes: a seeded person gets a row when they have a week or any extra dates; a missing week is stored as null (follows the business). Proved by planting a person with only an extra date: their row was written with no week and the date; the plant was removed after. Awaiting re-review. Closed 2026-09-27 by /audit independent (step 2.4, e58e0a0): `seed-dev.ts:340` now skips a row only when there is no week and no extra dates, and `:356` stores a missing week as `null` (follows the business), which the person branch of the 2.1 schema accepts. An entry with only `dateHours` now gets its row; Daniel's explicit `weeklyHours: null` still works. No new defect.

2/F-24 [P3] closed - The seed's types are not exported, and `PersonSeedType` also describes rooms

File: packages/shared/scripts/seed-dev.ts:57 **Found:** 2026-09-26 by /audit independent (scope: current; lens: quality) **Why it matters:** `coding-standards.md` (Naming) says types end in `Type` "and always exported", the rule F-20 enforced one step earlier; `PersonSeedType` (57), `ServiceSeedType` (64) and `TransactionType` (200) are all unexported. The same section says names say what a thing is: `PersonSeedType` carries `kind: "person" | "place"` and types the five rooms (lines 176-180), while the schema calls both a resource and the helper beside it is `ensureResource`. **Suggested fix:** Export the three types and rename `PersonSeedType` to `ResourceSeedType` (and `people` to `resources` if wanted). **Resolution:** 2026-09-26, fixed by /implement on Frank's yes: `ResourceSeedType` (renamed from `PersonSeedType`, since it also describes rooms), `ServiceSeedType` and `TransactionType` are exported. The seed type-checks on its own. Awaiting re-review. Closed 2026-09-27 by /audit independent (step 2.4, e58e0a0): `ResourceSeedType` (:68), `ServiceSeedType` (:75) and `TransactionType` (:215) are exported, and no `PersonSeedType` remains anywhere in the repo's code. The `people` key was left as is, which the suggested fix made optional.

2/F-25 [P3] closed - On a dev database that was not rebuilt, the seed adds the new businesses beside the old ones

File: packages/shared/scripts/seed-dev.ts:247 **Found:** 2026-09-26 by /audit independent (scope: current; lens: quality) **Why it matters:** The seed finds businesses by slug and never touches others. On any machine whose `scheduleleads_dev` still holds `agency-dev` and `test-salon-dev` (every machine but this one, which was rebuilt for 2.3), `db:seed` creates `painting-dev` and `clinic-dev` and a second owner membership for each dev account, so both accounts land in the pick-a-business state and the

admin's home may open on the old, empty agency. Step 2.5's Done when ("the home lists `painting-dev` 's three links") then depends on which business is active. Nothing in `AGENTS.md` (Commands) or the seed header says an existing dev database must be rebuilt after this step. **Suggested fix:** One line in `AGENTS.md` Commands and the seed header: after pulling 2.3, drop and rebuild the local `scheduleads_dev` (`db:migrate` , then `db:seed`). Removing the old businesses from the seed is not needed. **Resolution:** 2026-09-26, fixed by /implement on Frank's yes (option A, the seed cleans up): the seed deletes the retired made-up businesses `agency-dev` and `test-salon-dev` by exact slug, inside the local `_dev` guard, and says so; their members, people, hours and services cascade. Proved by putting both back with members, a person and a service: the seed removed them, left exactly the new cast, and the admin login belongs to one business again; a second run removed nothing. Awaiting re-review. Closed 2026-09-27 by /audit independent (step 2.4, `e58e0a0`): the delete (`seed-dev.ts:246-250`) runs inside the seed's one transaction, matches only the two exact slugs in `RETIRED_DEV_SLUGS` (`:89`), and can only be reached after `assertLocalDevelopmentDatabase` (`:238`) has accepted a loopback host and a `_dev` name at module load, before any connection is made. Members, resources, hours and services cascade from `organization` in the schema, so no orphan is left. No new defect.

2/F-26 [P3] closed - The subscription middleware still says it is the only place that reads `organization.plan`

File: backend/middleware/subscription-middleware/require-known-subscription-middleware.ts:27

Found: 2026-09-27 by /audit independent (scope: current; lens: quality) **Why it matters:** The comment reads "The only place in the API that reads `organization.plan`, so an unknown value is handled one way, in one place." Step 2.4 made that false: `findBookableOrganizationId` in `backend/routes/public-booking-links-routes.ts:40` and `:49` reads the plan too, and handles an unknown value differently on purpose (the identical `404` , not `403 plan_unrecognised`). Both behaviours are right, but a reader changing plan handling in feature 23 is told there is one place to change, and would miss the public one.

Suggested fix: Reword the comment to name both readers and why they answer differently: the dashboard says `plan_unrecognised` , the public routes say nothing is here. **Resolution:** 2026-09-27, fixed on Frank's yes: the comment at `require-known-subscription-middleware.ts:27` now names both readers of the plan and why they answer differently (the dashboard's `403 plan_unrecognised` , the public routes' identical `404`), and `findBookableOrganizationId` in `public-booking-links-routes.ts` points back to it. Comments only; 30 backend tests pass. Awaits the next review to close. Closed 2026-09-27 by /audit independent (step 2.5, `78370d9`): `require-known-subscription-middleware.ts:27-30` names both readers of the plan and why they answer differently, and `public-booking-links-routes.ts:37-38` points back to it. Both read it through the shared subscription config, so an unknown plan is `LOCKED` and falls under the comment's "plan without booking". No other reader of `organization.plan` exists under `backend/` . Comments only, no new defect.

2/F-27 [P3] closed - The coding standards were not updated for the first public route

File: blueprint/context/coding-standards.md:178 **Found:** 2026-09-27 by /audit independent (scope: current; lens: quality) **Why it matters:** Three lines in the standard now describe the backend as it was

before 2.4:

- :178-180 says `organizationId` "is derived server-side from the Better Auth session, never read from anything a client sends". The public routes take the business from the URL slug, which the spec approves as the one exception (`current-feature.md` , Notes for the AI, "Tenant scope"). That exception, and its conditions (read-only, find the business by slug first, then filter every query on its id, never a row by id alone), live only in the spec, which `/complete` archives. Features 5 and 9 add more public routes, including the first public write, and will read the standard, not the archive.
- :113-115 lists "Today's areas" without `middleware/public-middleware` .
- :36-39 says `server.ts` , `lib/` and `middleware/` sit in `backend/` ; it now also holds `app.ts` and a new kind folder, `routes/` , which the kind-then-area rule at :98-104 does not mention. **Suggested fix:** Add the public-route exception with its conditions beside the tenant rule, add `middleware/public-middleware` to the areas list, and name `app.ts` and `routes/` in the backend layout. **Resolution:** 2026-09-27, fixed on Frank's yes: `coding-standards.md` now states the public-route exception beside the tenant rule, with its four conditions (read-only, business by slug first then every query on its id, never a row by id alone, one identical 404); names `app.ts` , `server.ts` and `routes/` in the backend layout; adds `routes/<area>-routes.ts` to the kind-then-area rule; and lists `middleware/public-middleware` and `routes/public-booking-links-routes.ts` among today's areas. Awaits the next review to close. Closed 2026-09-27 by /audit independent (step 2.5, 78370d9): `coding-standards.md:197-205` states the public-route exception beside the tenant rule with its four conditions, and they match the code (`public-booking-links-routes.ts:39-52` finds the business by slug first, :63-67 and :83-93 filter on its id and never find a link by id alone, one `notFound` body); :37-40 names `app.ts` , `server.ts` and `routes/` ; :111 adds `routes/<area>-routes.ts` ; :124-125 lists `middleware/public-middleware` . No new defect from the repair; an unrelated wording slip that step 2.5 introduced in the same file is F-30.

2/F-28 [P3] closed - Backend tests run against whatever `packages/shared/dist` holds, and silently need a seeded local Postgres

File: `backend/package.json:10` **Found:** 2026-09-27 by /audit independent (scope: current; lens: tests)

Why it matters: Until 2.4 the backend tests only imported shared types, which are erased. `public-booking-links-routes.test.ts` now loads `@scheduleads-app/shared/db` , `/subscriptions` and `/zod-validation` at runtime, and the package exports point only at `dist/` (`packages/shared/package.json` ; there is no Vitest alias in `backend`). `predev` and `prebuild` rebuild `dist/` first, but `test` has no `pretest` , so after a change to a shared schema `npm run test -- workspace=backend` passes or fails against the previous build, and on a fresh clone it fails on a missing module. Separately, `AGENTS.md` (Commands, :646) lists the backend test command with no word that it now needs the local `scheduleads_dev` running and seeded, which the spec's 2.4 Done when states; the test's own error only names the seed once the connection succeeds. **Suggested fix:** Add `"pretest": "tsc -p ../packages/shared/tsconfig.build.json"` (and the same before `test:watch` , or note it), and add one line under the backend test command in `AGENTS.md` : needs local Postgres with `db:migrate` and `db:seed` run. **Resolution:** 2026-09-27, fixed on Frank's yes: `backend/package.json`

has `pretest` and `pretest:watch` rebuilding `packages/shared` first, like `predev` and `prebuild`; `AGENTS.md` (Commands) says the backend tests need the local Postgres migrated and seeded, and fail rather than skip without it. Proved by deleting `packages/shared/dist` and running `npm run test -- workspace=backend`: the pretest rebuilt it and all 30 tests passed. Awaits the next review to close. Closed 2026-09-27 by /audit independent (step 2.5, 78370d9): `backend/package.json:16` and `:18` add `pretest` and `pretest:watch` (npm runs a `pre` script for any script name); `npm run test -- workspace=backend` was seen running `pretest` before Vitest, and 30 tests passed against the seeded local database. `AGENTS.md:660-665` says the tests need the local Postgres migrated and seeded and fail rather than skip. No new defect.

2/F-29 [P2] closed - A business with no hours yet shows "unexpected status (404)" and a Try again that cannot help

File: `frontend/lib/api-client.ts:82` **Found:** 2026-09-27 by /audit independent (scope: current; lens: quality) **Why it matters:** `fetchBookingLinks` treats every non-200 as unreachable, on the premise in its comment (`:82-84`) that a 404 "means the API and the dashboard disagree about this business". Step 2.4 made that false: the public list answers the identical 404 for a business with no business hours row (`public-booking-links-routes.ts:50` , tested at `public-booking-links-routes.test.ts:172-177`), and nothing but the dev seed writes that row until feature 12. Every business made through the app's own path (`/create-organization` , or the API as step 2.1's Done when did) gets its first person from the Better Auth hook (`auth-server.ts:114-120`) but no hours row, so its dashboard home shows "The API answered with an unexpected status (404)." with a Try again button (`booking-links-list.tsx:61-69`) that can never succeed. That is what `fetchMe` 's own comment (`api-client.ts:63-64`) says not to do: tell the user to do something that cannot help. The four real tenants will be created this way. Read off the code, not observed live (no dev server was started in this review). **Suggested fix:** Give the 404 its own state in `BookingLinksResultType` , shown without Try again as "not open for online booking yet" (from a slug `/me` just returned, its causes are no business hours or a plan without booking), and correct the comment at `:82-84` . Add the state to the spec's step 2.5 piece 4 list. **Resolution:** Fixed 2026-09-27 on Frank's yes: `fetchBookingLinks` returns a `not-bookable` state for the public 404 (`frontend/lib/api-client.ts`), shown by `booking-links-list.tsx` as "Not open for online booking yet." with no Try again; the wrong comment is gone and the spec's piece 4 lists the state. Proved live: a throwaway business with no hours, made active for the dev admin, showed the new message; removed after, Summit's three links back. Lint, frontend build and format check pass. Awaiting the next review to close. Closed 2026-09-28 by /audit independent (step 2.6, 5a30d47): `frontend/lib/api-client.ts:76-79` adds `{ state: "not-bookable" }` to `BookingLinksResultType` , `:91-93` returns it for a 404 with a comment that now states the real causes, and `booking-links-list.tsx:71-77` shows "Not open for online booking yet." with no Try again, while `:60-69` keeps Try again only for `unreachable` . The spec's step 2.5 piece 4 lists the state. Frontend build and lint pass. No new defect from the repair (the live check was the builder's; this pass read the code).

2/F-30 [P3] closed - The docs around the new types build are half updated

File: blueprint/context/coding-standards.md:59 **Found:** 2026-09-27 by /audit independent (scope: current; lens: quality) **Why it matters:** Step 2.5 inserted the `AppType` sentences into the middle of the `packages/shared` paragraph (:51-59), so the next sentence, "It compiles to `dist/` and its subpath exports point there", now follows "The frontend's `predev` and `prebuild` write them fresh" and reads as if it meant the backend's declarations; the inserted text is also left as one unwrapped line. Separately, `AGENTS.md` Commands (:644-647) still says both apps' `predev` / `prebuild` build only `packages/shared` , and does not list `npm run build:types --workspace=backend` or say that the frontend's dev server and build now compile every backend file `app.ts` imports, so a backend type error now stops `npm run dev --workspace=frontend` as well. Only the Settled architecture section (`AGENTS.md:74`) mentions it. **Suggested fix:** Move the `AppType` sentences after the `packages/shared` paragraph (or start the next sentence with " `packages/shared` compiles"), rewrap it, and add one line to `AGENTS.md` Commands naming `build:types` and that the frontend's `predev` / `prebuild` run it. **Resolution:** Fixed 2026-09-27 on Frank's yes: in `coding-standards.md` the `packages/shared` paragraph is whole again, so "It compiles to `dist/` " follows the sentences about shared, and the `AppType` sentences are their own paragraph after it, rewrapped. `AGENTS.md` Commands now says the frontend's `predev` and `prebuild` also run `npm run build:types --workspace=backend` , so a backend type error stops the frontend's dev server and build too. Docs only, no code touched. Awaiting the next review to close. Closed 2026-09-28 by /audit independent (step 2.6, 5a30d47), read at the target commit: `coding-standards.md:51-54` is the whole `packages/shared` paragraph, so "It compiles to `dist/` " now follows the sentences about shared, and the `AppType` sentences are their own wrapped paragraph from :62 . `AGENTS.md:647-651` (as committed) says the frontend's hooks also run `npm run build:types --workspace=backend` and that a backend type error stops the frontend's dev server and build. No new defect.

2/F-31 [P3] closed - Holiday dates are recomputed on every public request, with work that grows with an unbounded horizon

File: backend/lib/bookable-hours/closed-holidays.ts:45 **Found:** 2026-09-28 by /audit independent (scope: current; lens: performance) **Why it matters:** `closedHolidayDates` builds a fresh catalogue and calls `getHolidays(year)` on two lists for every year the horizon touches, on every call, and `GET /public/:slug/booking-links/:bookingLinkId` calls it on every request (through `resolveBookableHours`). The work is synchronous CPU on the event loop of a public, unauthenticated, not rate-limited route. Measured against the built backend: about 1.0 ms per call for a 60-day horizon, 1.6 ms across New Year, 7.5 ms for ten years and 68 ms for a hundred. `horizonDays` has no upper bound in the database check (`> 0`) or the shared schema (`z.int().min(1)` , `availability-rule-validation-schema.ts:26`), so from feature 12, when owners write their own row, one business with a large horizon makes each public request to it cost tens of milliseconds of blocked CPU. At today's 60 and 120 days it is harmless; before step 2.6 the horizon's size cost nothing. **Suggested fix:** Give `horizonDays` a sensible maximum in the shared schema and the database check (for example 365 or 730) when feature 12 adds its writer, and optionally cache each list's holidays per year in the module (the lists are already cached; the years are not). **Resolution:** Fixed 2026-09-28 on Frank's call: customers can book at most a year

ahead. `availability_rule_horizon_check` is now `horizonDays between 1 and 365` (migration `0003_horizon_at_most_a_year`) and the shared schema has `.max(365)`, with a saved test (365 accepted, 366 refused) that fails with the maximum removed; 366 was refused by the database by hand, in a rolled-back transaction. Measured on the built backend with all nine holidays picked: about 1 ms for 30 and 60 days, 1.5 ms for 120 and 365, so the cap bounds the cost. Per-year caching was not added: at a year it would save about a millisecond. The one-month starting value belongs to feature 12's settings screen, pre-filled for the owner to change. For the next review to close. Closed 2026-09-28 by /audit independent (feature 2 final review, `9ff399f`): `availability-rule-table.ts:88` declares `horizonDays between 1 and 365`, migration `0003_horizon_at_most_a_year.sql` drops and re-adds the check, and the `0003` snapshot carries the same value. On the local `scheduleads_dev` the migration ledger holds four entries, the live constraint reads `horizonDays >= 1 AND horizonDays <= 365`, and in a rolled-back transaction 366 was refused and 365 accepted (both business rows back at 60 and 120 afterwards). The shared schema has `.max(365)` (`:26`) and the saved test at `availability-validation-schemas.test.ts:131` accepts 365 and refuses 366, so removing the maximum flips it; 30 shared tests pass. The cap bounds the per-request year loop in `closed-holidays.ts:45` to at most two years. No new defect: only local seeded rows exist (60 and 120 days), so the new check cannot fail to apply on them.

2/F-33 [P3] closed - The spec still says step 2.6's plan is "not agreed yet" and records neither its approval nor what was built

File: `blueprint/context/current-feature.md:327` **Found:** 2026-09-28 by /audit independent (scope: current; lens: quality) **Why it matters:** Step 2.6 is ticked `[x]` and built in `5a30d47`, but its text still reads "Plan, rewritten 2026-09-28 (not agreed yet)", and unlike steps 2.1 to 2.5 it has no "Approved by Frank" or "Built" line, so the only record in the repo says the plan was never agreed. The approval and the by-hand checks live only on the build log page. The Files / areas list is also stale: `:395` names `backend/routes/public-booking-links.ts` (the file is `public-booking-links-routes.ts`) and it omits `backend/lib/bookable-hours/closed-holidays.ts` and migration `0002_closed_holidays.sql`. The workflow rules say a spec found wrong in review is corrected before the next step builds on it. **Suggested fix:** Replace "(not agreed yet)" with the approval and its date, add a short "Built 2026-09-28" line like 2.4's (the lockfile gained `date-holidays` and twelve helpers; 43 MB on disk), and correct the Files / areas list. **Resolution:** Fixed on Frank's yes, 2026-09-28. Step 2.6 now reads "Plan, rewritten 2026-09-28" with an "Approved by Frank, 2026-09-28" line, a "Built 2026-09-28 (`5a30d47`)" line naming the added country check and the tests, and a line for the review fixes (F-31, F-32, F-33). The Files / areas list names the three migrations, `public-booking-links-routes.ts` and its test, the public CORS middleware, `closed-holidays.ts`, `tsconfig.types.json` and `booking-links-list.tsx`, each checked to exist. The stale Notes line "2.6 waits for Open question 2" was removed. Docs only. For the next review to close. Closed 2026-09-28 by /audit independent (feature 2 final review, `9ff399f`): `current-feature.md:328` reads "Plan, rewritten 2026-09-28" with no "not agreed yet", `:385-393` add the approval, the built line (`5a30d47`, the added country check, the tests) and the review fixes, and the Files / areas list (`:395-416`) names `public-booking-links-routes.ts` and its test,

`closed-holidays.ts` and the three migrations; every path it names exists at the target. No "waits for Open question 2" line remains. No new defect.

Independent review

Status: passed **Target commit:** `a29bfc0420d39c72c0dacf3841e52a663d4e53c6` **Base commit:** `931ecadd49bebb38bd9f02bf27a2deeab0236755` **Base ref:** main **Spec hash:** `79b0313c7bdbaea3b0951e24d15d406cd2ea90fdbc16676f69c66febf81dac26` **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-09-28T16:59:49Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-09-28T17:02:00Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete `931ecad..a29bfc0` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

This is `/complete`'s final review of feature 2, repeated because the target moved. A fresh reviewer passed the whole feature at `9ff399f` (same base, same spec hash) on 2026-09-28T16:56Z; that receipt went stale only because one more commit landed on the branch while it ran:

- `a29bfc0`: `blueprint/context/project-log.html` only, the build log page (not product code, not served by either app). CSS makes a feature's drawers read as buttons; each feature's "why" paragraph moves from above the gates into its "Why this item exists" drawer (items 1, 2 and the template); the page script's `jumpTo` centres a short step between the top bar and the bottom of the screen, and a step opened by tapping its title glides there.

The spec is byte-for-byte unchanged since `9ff399f`, and no file under `frontend/`, `backend/` or `packages/` changed in `9ff399f..a29bfc0`. The earlier receipt's evidence for `931ecad..9ff399f` is saved outside the repo; confirm that range's product code is unchanged, rerun the tests, builds, lint and format check, and give the new page commit its own full read (script errors, anything loaded from outside `cdn.jsdelivr.net`, content that contradicts the spec).

Commands

- `git rev-parse HEAD`, `git merge-base main HEAD`, `sha256sum blueprint/context/current-feature.md`, `git status --porcelain --untracked-files=all`: pass (HEAD `a29bfc0`, merge base `931ecad` and spec hash match the request; only `review.md` and `findings.md` differ from the target)
- `git diff --stat 9ff399f a29bfc0` and `git diff --quiet 9ff399f a29bfc0 -- frontend backend packages package.json .prettierrc .prettierignore`: pass (only

`blueprint/context/project-log.html` changed, +39/-8; product code identical to the reviewed `9ff399f`)

- `npm run test --workspace=@scheduleads-app/shared` : pass (2 files, 30 tests)
- `npm run test --workspace=backend` : pass (pretest rebuilt shared; 2 files, 38 tests, against the local seeded `scheduleads_dev`)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass (prebuild ran the shared build and `build:types` ; 5 static routes)
- `npm run lint --workspace=frontend` : pass
- `npm run format:check` : pass
- `node --check` on every inline classic and module script of `project-log.html` at the target (8 scripts): pass; the ninth inline block is the `code-theme` JSON data, not a script
- `/check` : not run (Check not required)

Evidence

- Whole feature range `931ecad..9ff399f` : product code confirmed byte-identical at `a29bfc0` , so the prior fresh receipt's full read of that range (tenant boundary of the public routes, public CORS, bookable-hours rules, holidays, F-31 database check, branch-deletion docs) still describes the code under review; its tests, builds, lint and format were rerun here and pass.
- `a29bfc0` page read in full (`project-log.html` diff): tag balance unchanged against `9ff399f` (`details` 341/341, `div` 1647/1647, `p` 312/312). The two item 1 paragraphs moved verbatim into item 1's own "Why this item exists" drawer (lines 5621-5622, inside item 1's fold that closes at 6311); item 2's paragraph moved verbatim into item 2's drawer (line 9731); the template's placeholder moved the same way. Items 0a, 0b and 3 to 26 keep `item-text` as one-line rows, so the `.item-text` rule is still used.
- New script: `jumpTo` reads `.topbar` (present once, line 1403), never scrolls under the bar (`bar + 12`), centres a `.rstep` only when it fits in `innerHeight - bar - 32` , and clamps to `[0, scrollHeight - innerHeight]` . The tap handler calls it through `setTimeout(0)` , after the summary's own open has run, and only for `.rstep` , so feature folds keep their previous behaviour; the existing wheel, touch and key cancel still stops the settle check.
- External loads: the commit adds none. The page still loads only `cdn.jsdelivr.net` scripts (mermaid, shiki) and the Google Fonts stylesheet it already had; the other URLs are text inside code drawers.
- Spec agreement: the moved item 2 paragraph matches the spec's Goal (three primitives, a public read-only route, one resolving function, `hc<AppType>` breaking the build). No moved or new text contradicts `current-feature.md` .
- F-34: `backend/tsconfig.types.json:2` , `backend/tsconfig.json:14` , `packages/shared/tsconfig.build.json:3` still carry the history asides; unchanged, stays open.

Findings

- None new
- F-34 [P3] open, unchanged (tsconfig comments carry step and date history)
- F-32 [P3] unverified, unchanged (carried to feature 12)
- F-31 [P3] and F-33 [P3] closed by the `9ff399f` pass; their repaired files are byte-identical at `a29bfc0`
- F-12 [P2], F-13, F-14, F-16 [P3] open from feature 1, not re-examined (their files are outside this delta); none blocks

Remaining risk

- Item 2's gate chips on the build log still read "Spec agreed" waiting with the other two blank (line 6319), while its Steps drawer says 2.1 to 2.6 are done and reviewed. Not introduced by `a29bfc0`; `/complete` Step 1b sets all three gates passed before the final commit, so it needs doing there, and that edit moves `HEAD` off this target.
- The new scroll behaviour was checked by reading and `node --check` only, not in a browser or on a phone.
- `/check` was not run (not required); the by-hand browser checks of the CORS block, the typed-seam broken build and the dashboard list states rest on the builder's step evidence.
- The person branch of `resolveBookableHours` (a `resourceId` given) has no saved test and no caller yet; it was proved by hand in step 2.2 and first gets used in feature 5.
- Public routes have no rate limit (spec, Out of scope) and a thrown holiday lookup answers `500` for that business (F-32, feature 12).
- No dependency vulnerability scan was run; none is declared, and manifest inspection is not a scan.